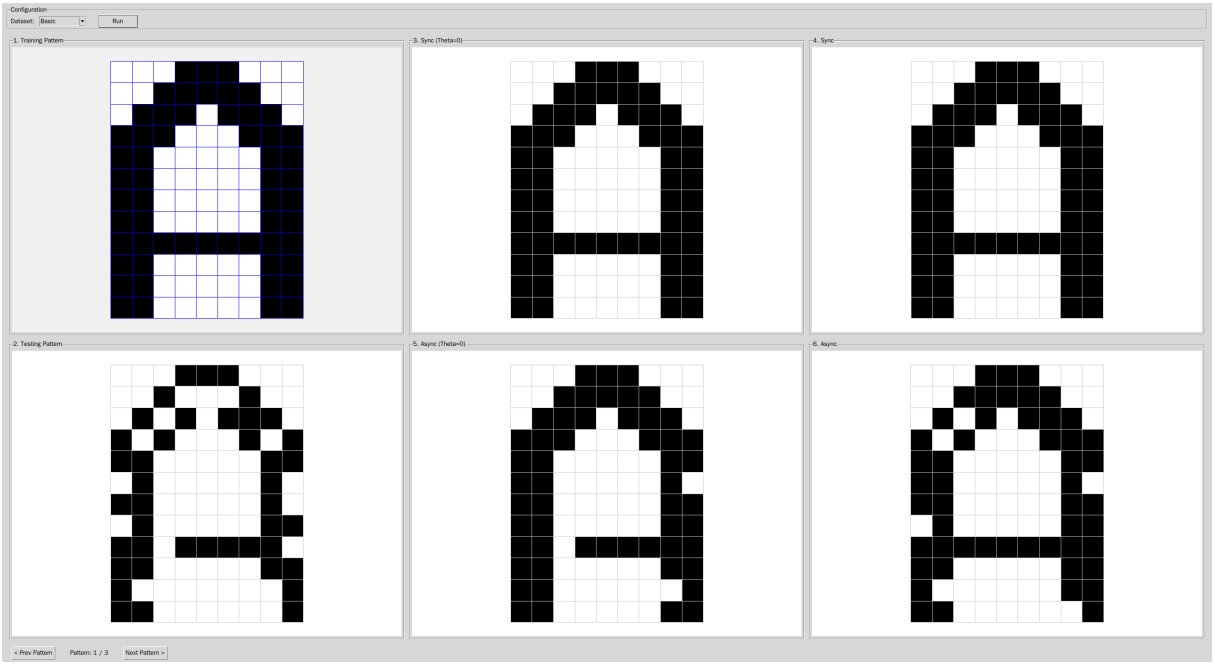
一 Hopfield Network

二 加分題完成功能

- [x] 使用 Bonus_Training.txt 和 Bonus_Testing.txt 資料集
- [x] 可選擇同步或非同步更新
- [x] 可選擇是否使用 Theta(=0)

三 程式執行說明 (GUI 功能說明)



- 左上角可以選擇 Basic 或 Bonus 資料集
- 左下角可一選擇 pattern
- 中間顯示結果(train, test, 及四種結果)

四 Hopfield 程式碼簡介

```
def train(self, patterns, threadhold=False):
  N = len(patterns)
  if N == 0:
    return

  for p in patterns:
    self.weights += np.outer(p, p.T)

  self.weights -= N * np.identity(self.pattern_size)
  self.weights /= self.pattern_size
  self.theta = np.sum(self.weights, axis=0).reshape(-1, 1) if threadhold else
np.zeros((len(self.weights), 1))
```

$W = \frac{1}{p}\sum_{k=1}^{N} x_k x_k^T - \frac{N}{P} I$

$\theta_j = \sum_{i=1}^{p} w_{ji}$ 或是 $\theta = 0$

```
def _update_unit(self, x, i=None):
  if i is not None:
    v = self.weights[i] @ x - self.theta[i]
```

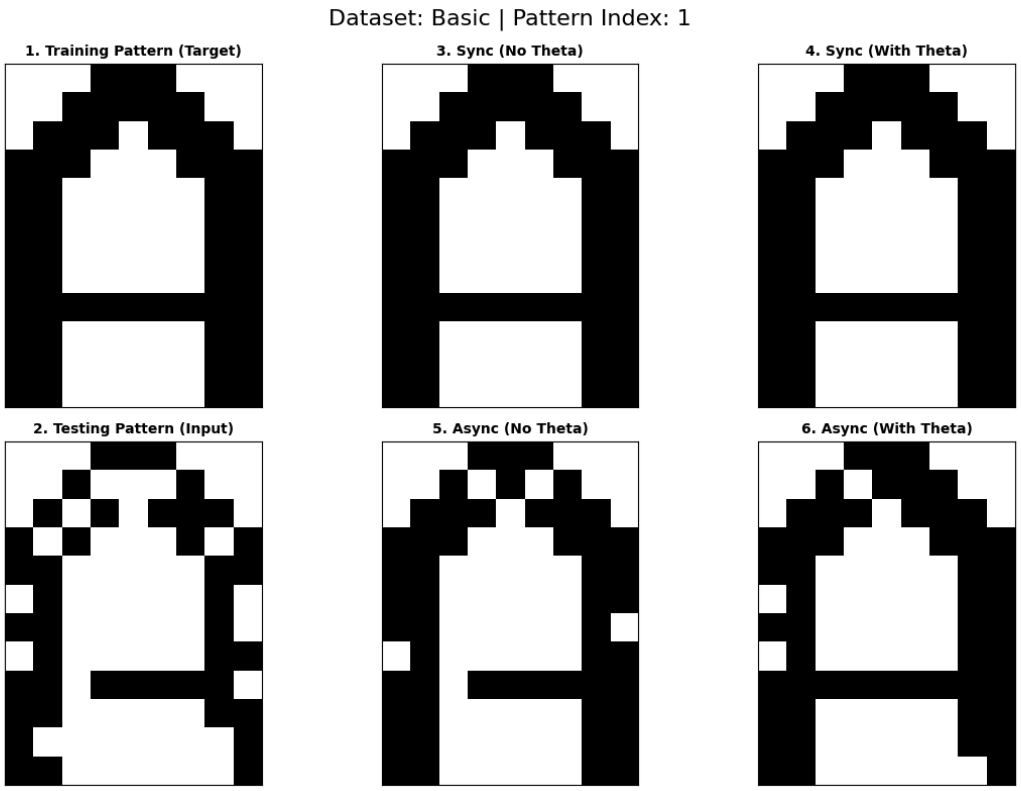
```
    else:
      v = self.weights @ x - self.theta
    for j in range(len(v)):
      if v[j] > 0:
        v[j] = 1
      elif v[j] == 0:
        v[j] = x[j]
      elif v[j] < 0:
        v[j] = -1
    return v

  def predict(self, pattern, max_iter=1000, asynchronous=False):
    x = pattern.copy()
    for iter in range(max_iter):
      old_x = x.copy()
      if asynchronous:
        i = np.random.randint(0, self.pattern_size)
        x[i] = self._update_unit(x, i)[0]
      else:
        x = self._update_unit(x)
      if np.array_equal(x, old_x) and iter >= 100:
        break
    return x
```

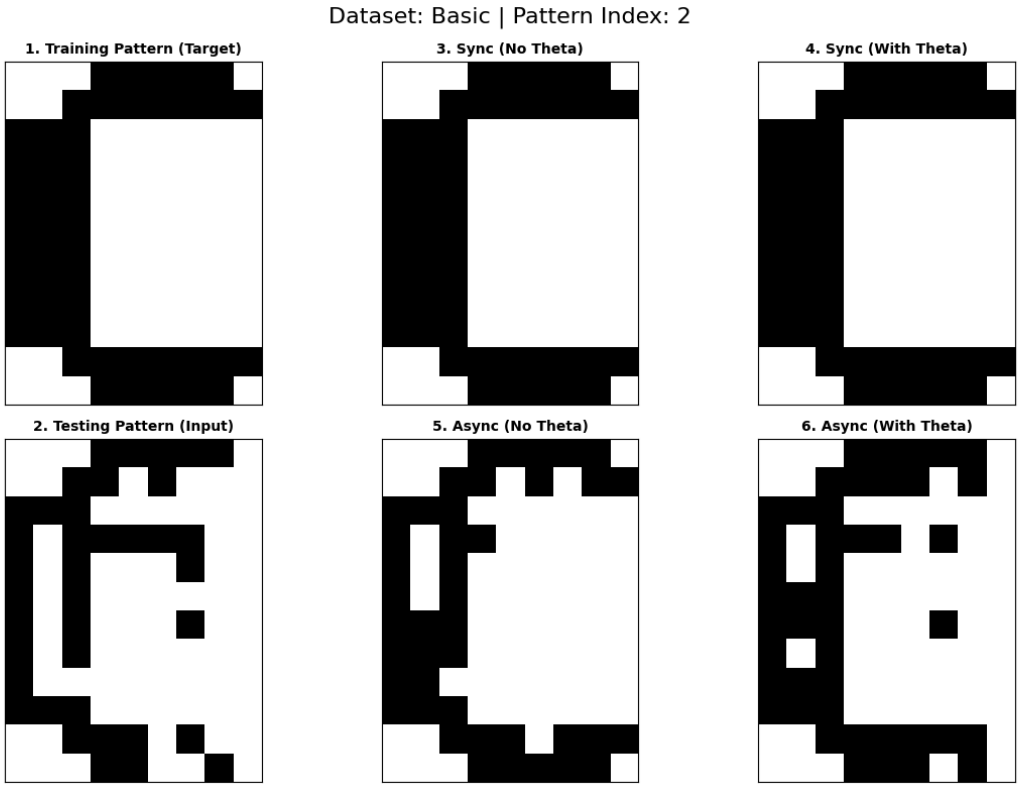
將是否同步分開處理，若為非同步每次更新一個 row

五 實驗結果

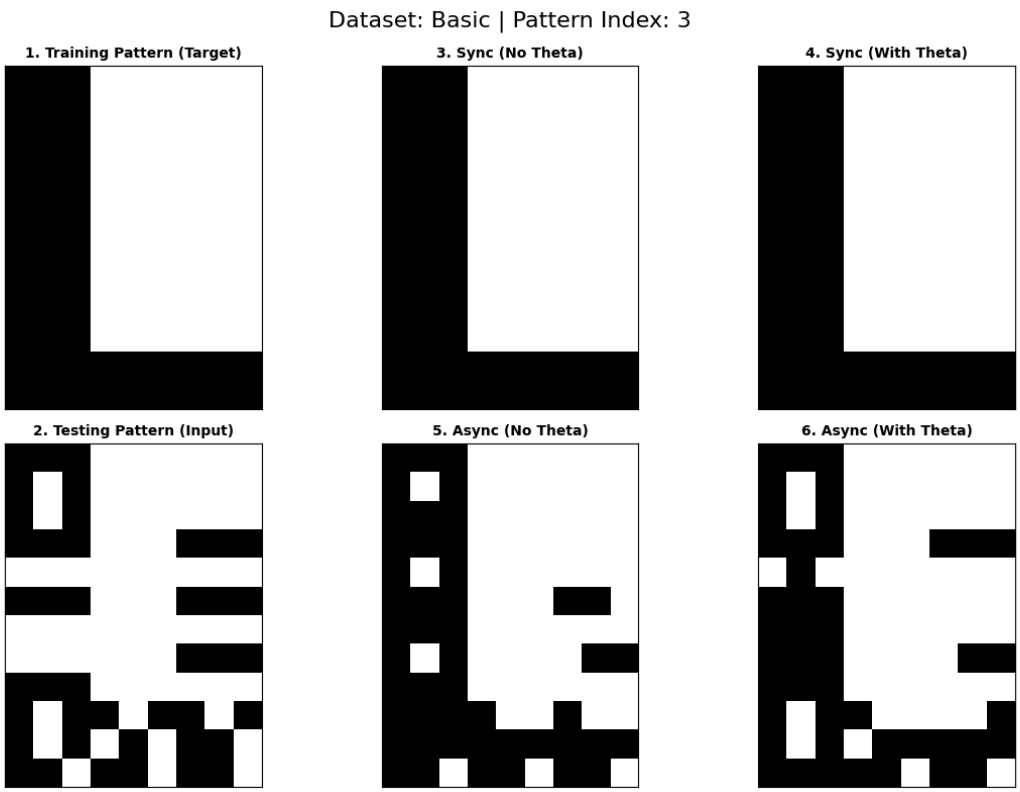
五.1 Basic



Basic 1

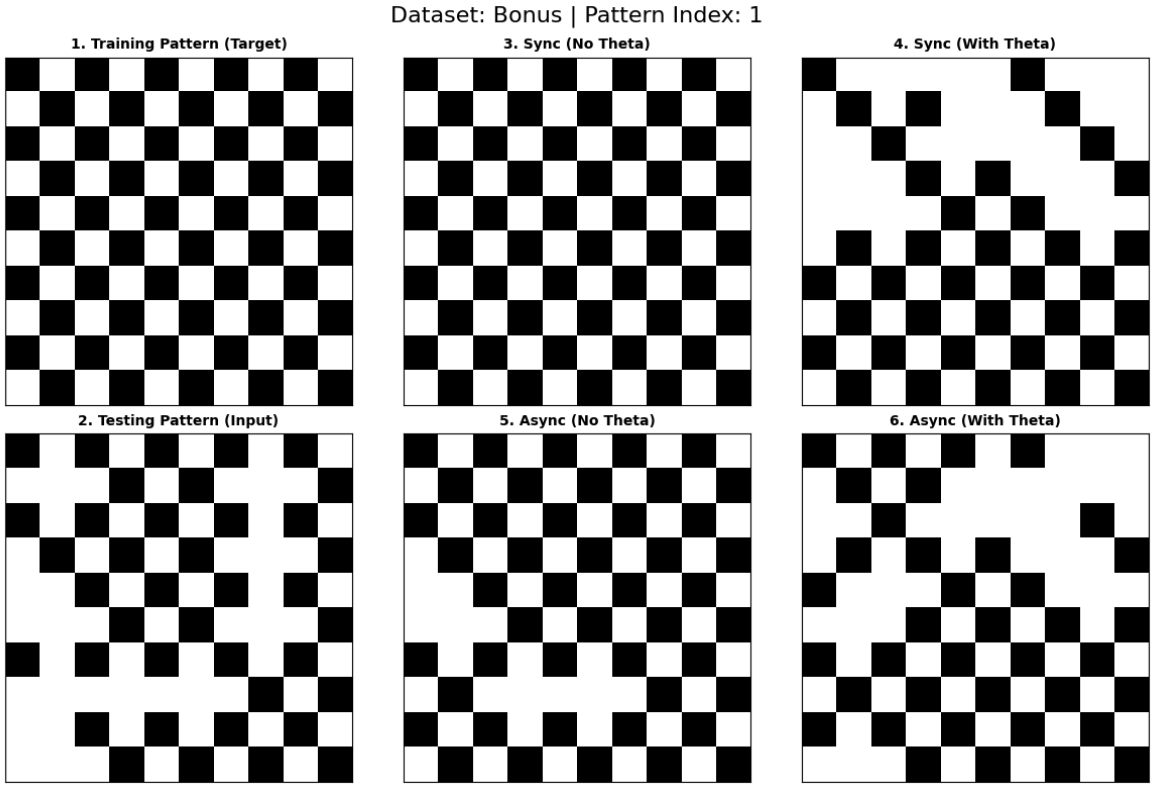


Basic 2

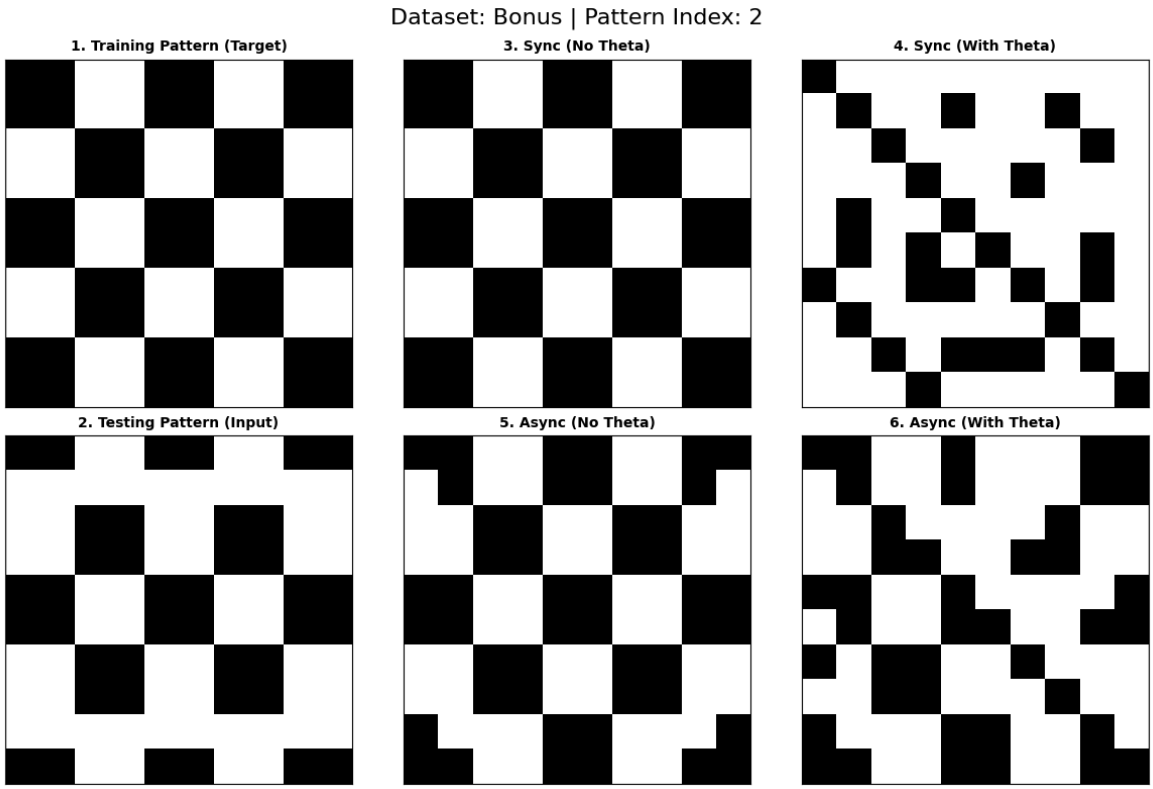


Basic 3

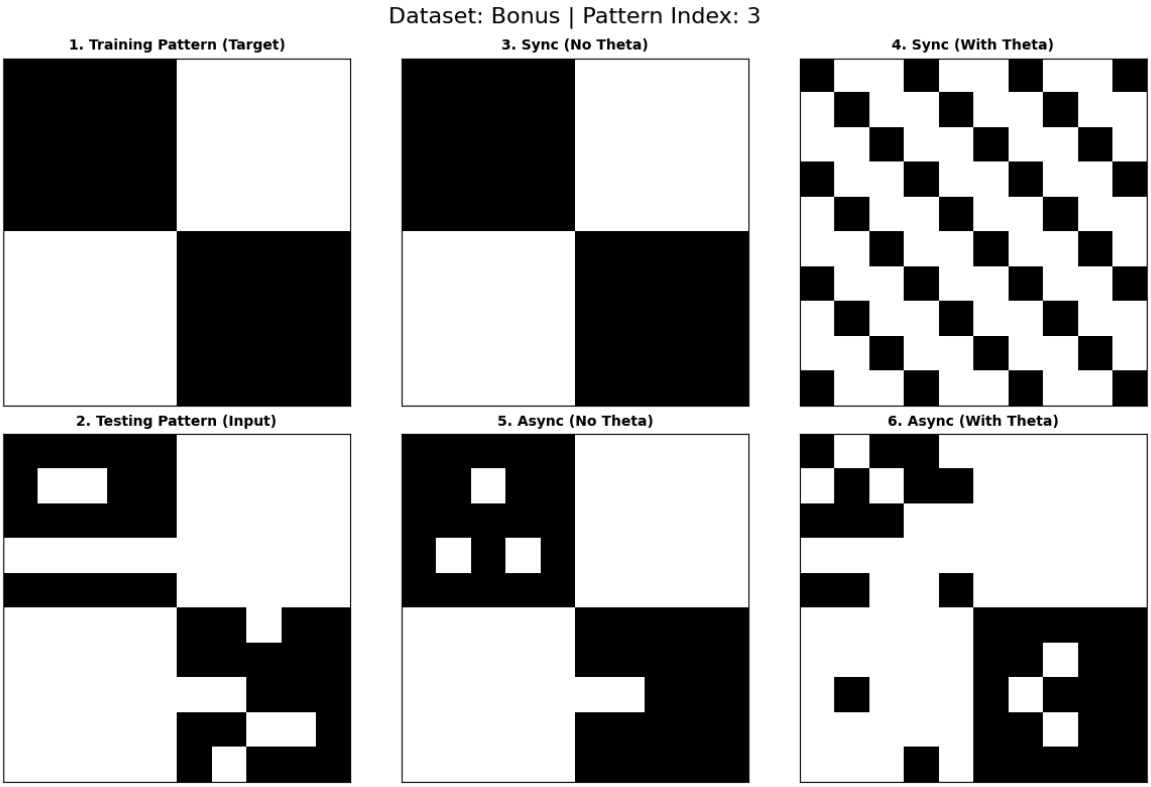
五.2 加分題



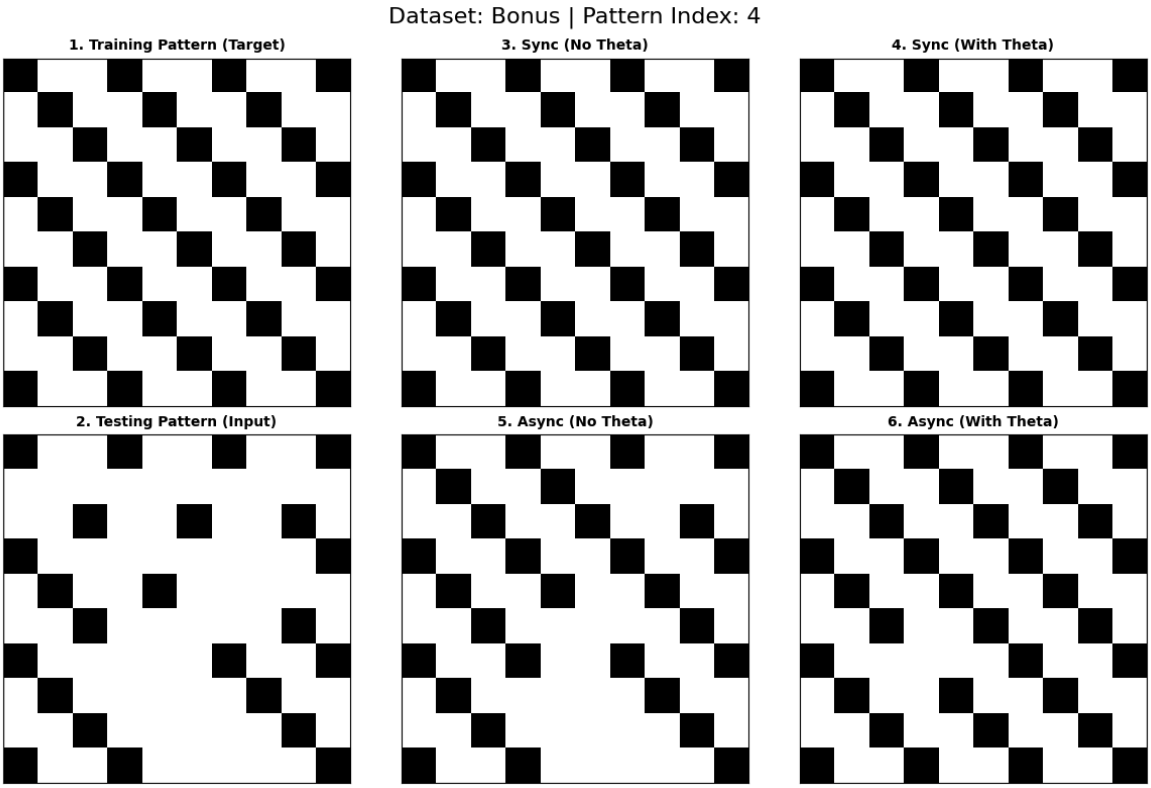
Bonus 1



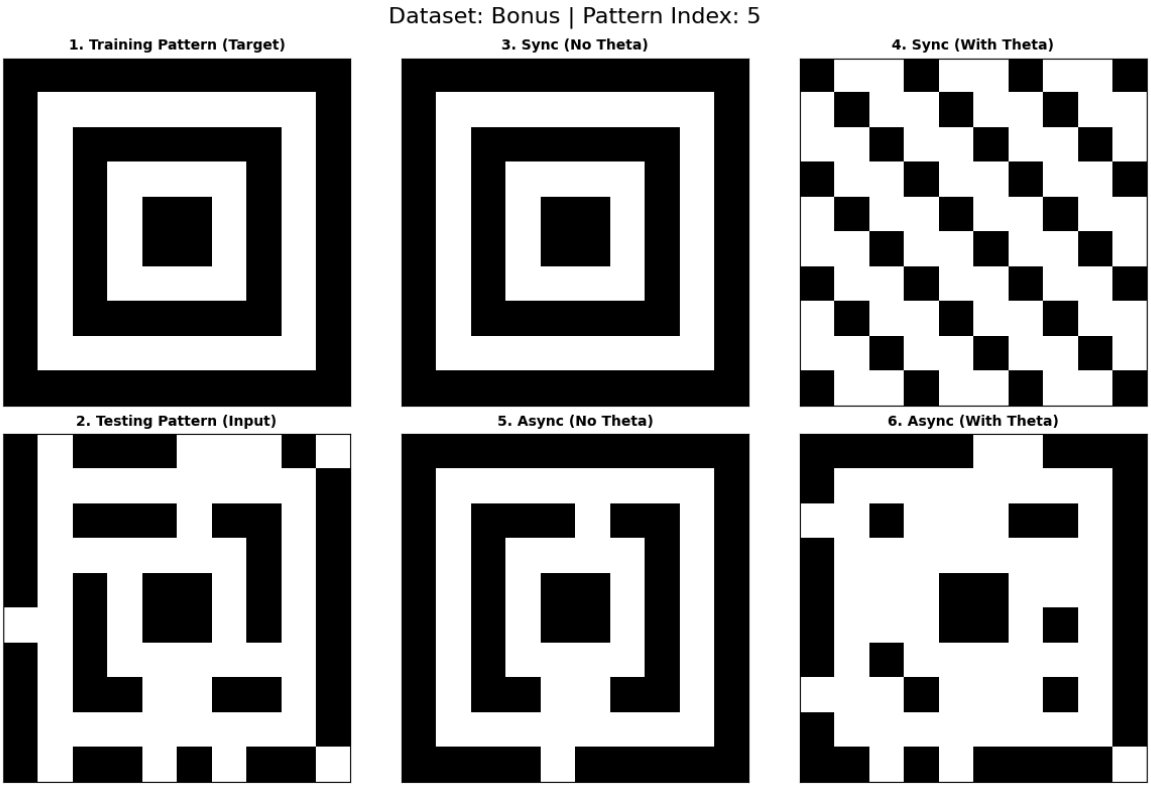
Bonus 2



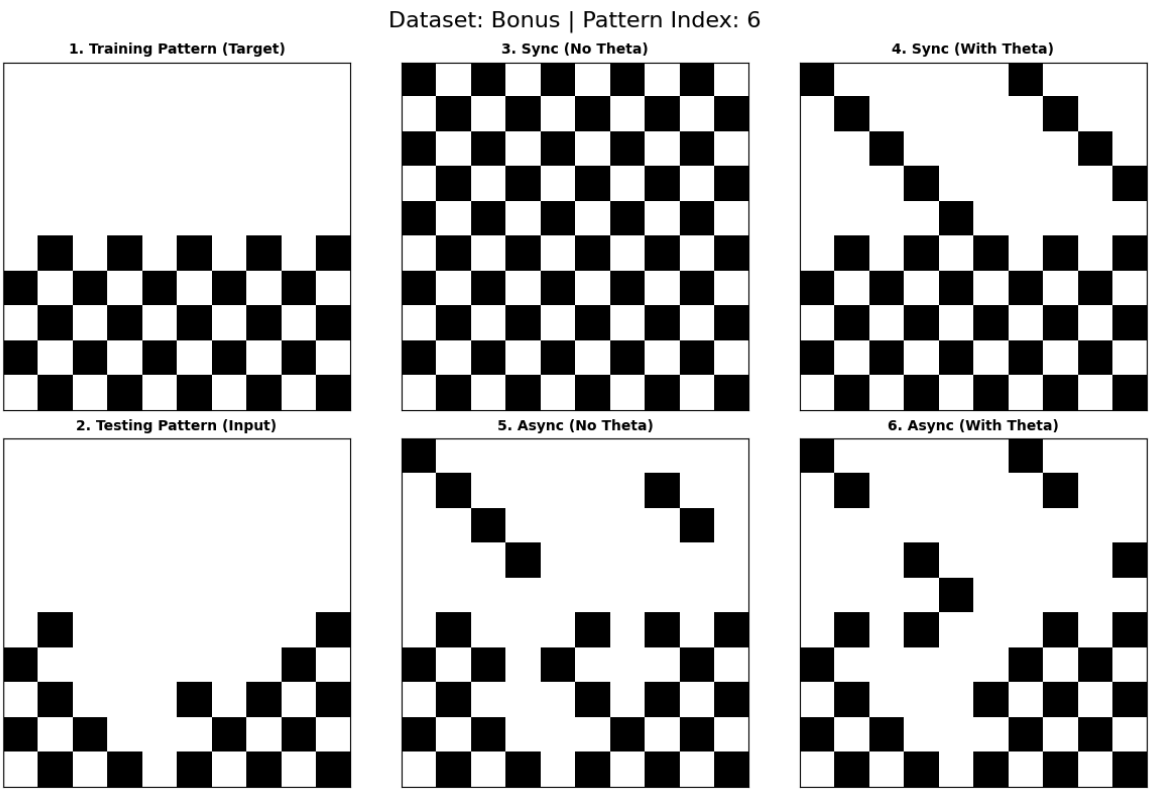
Bonus 3



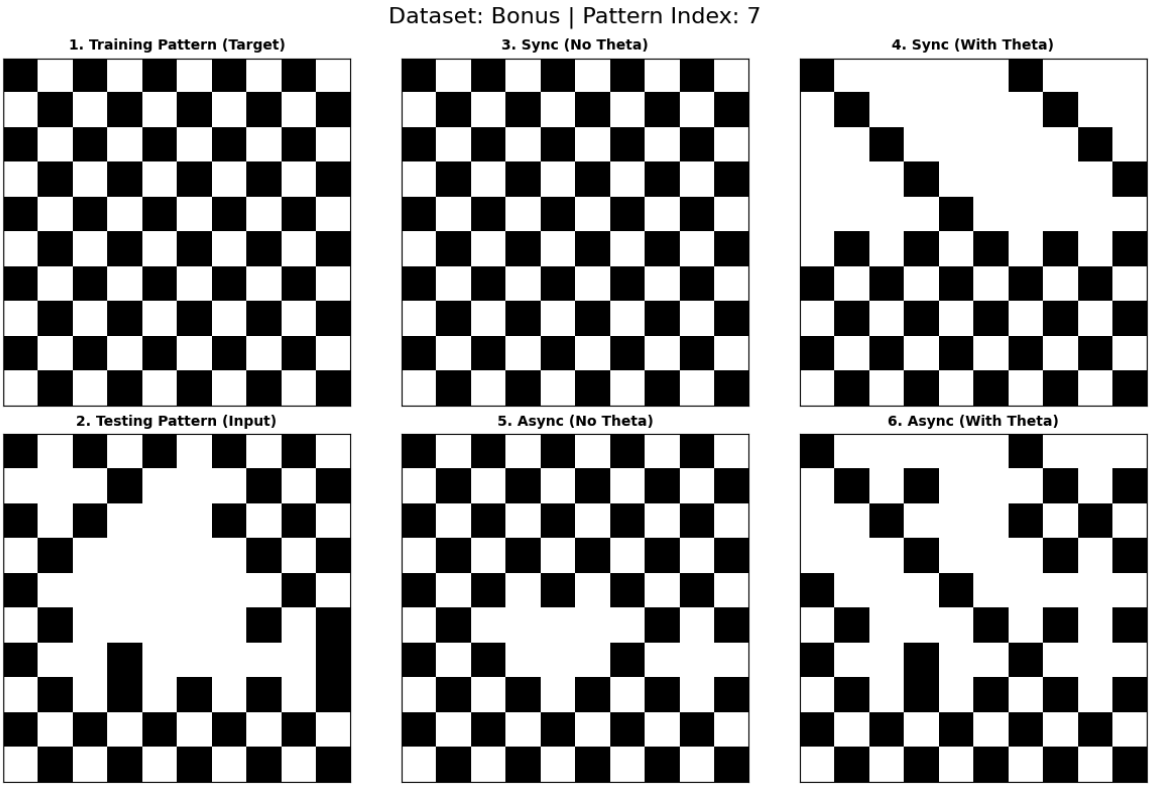
Bonus 4



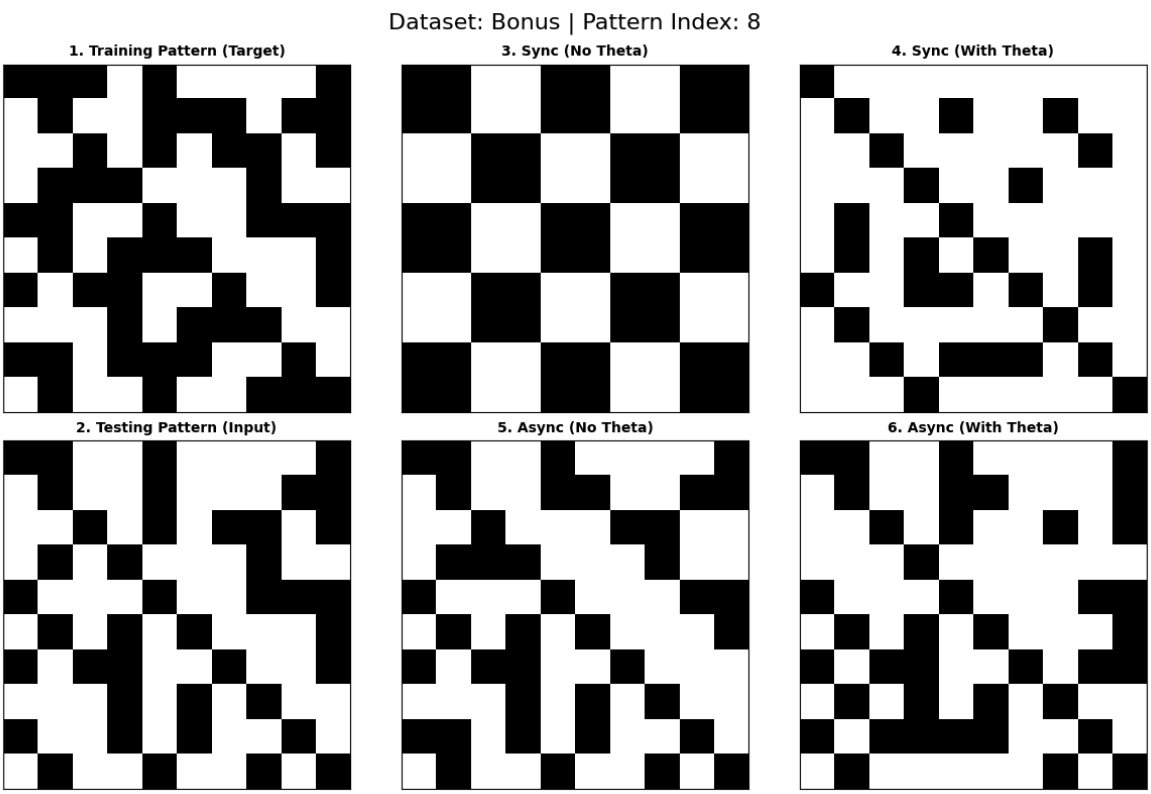
Bonus 5



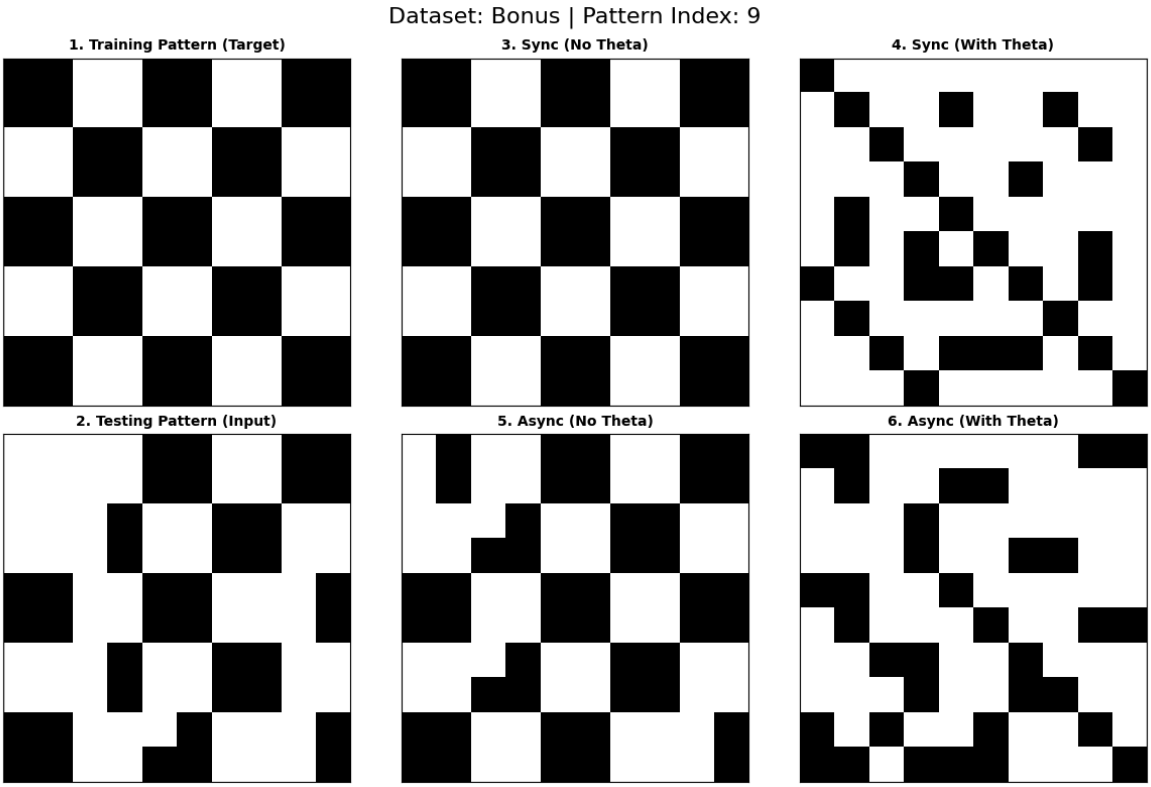
Bonus 6



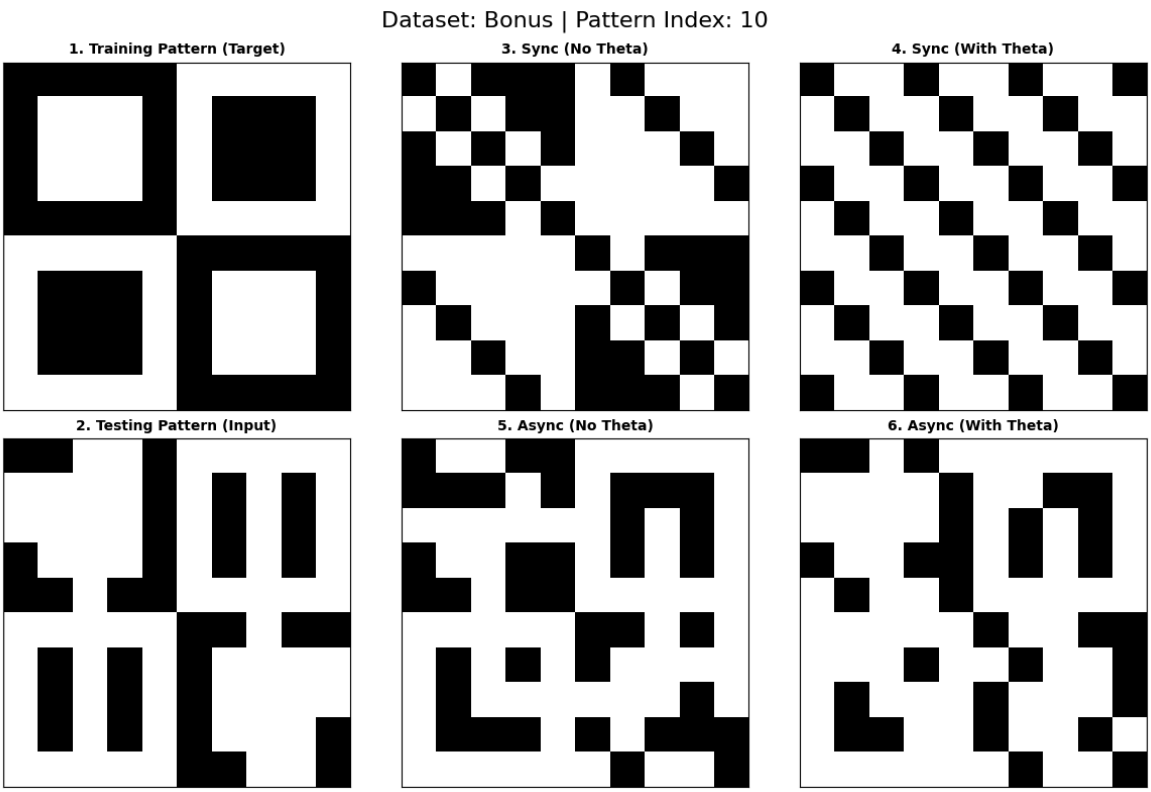
Bonus 7



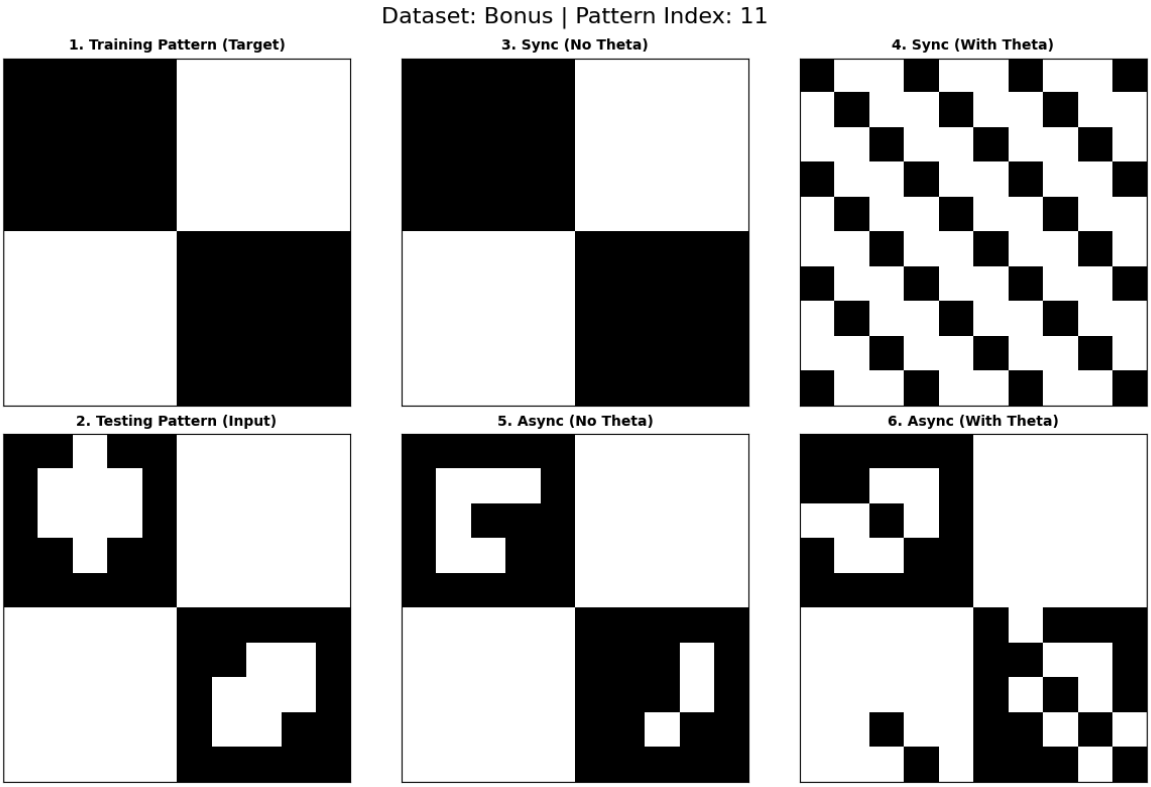
Bonus 8



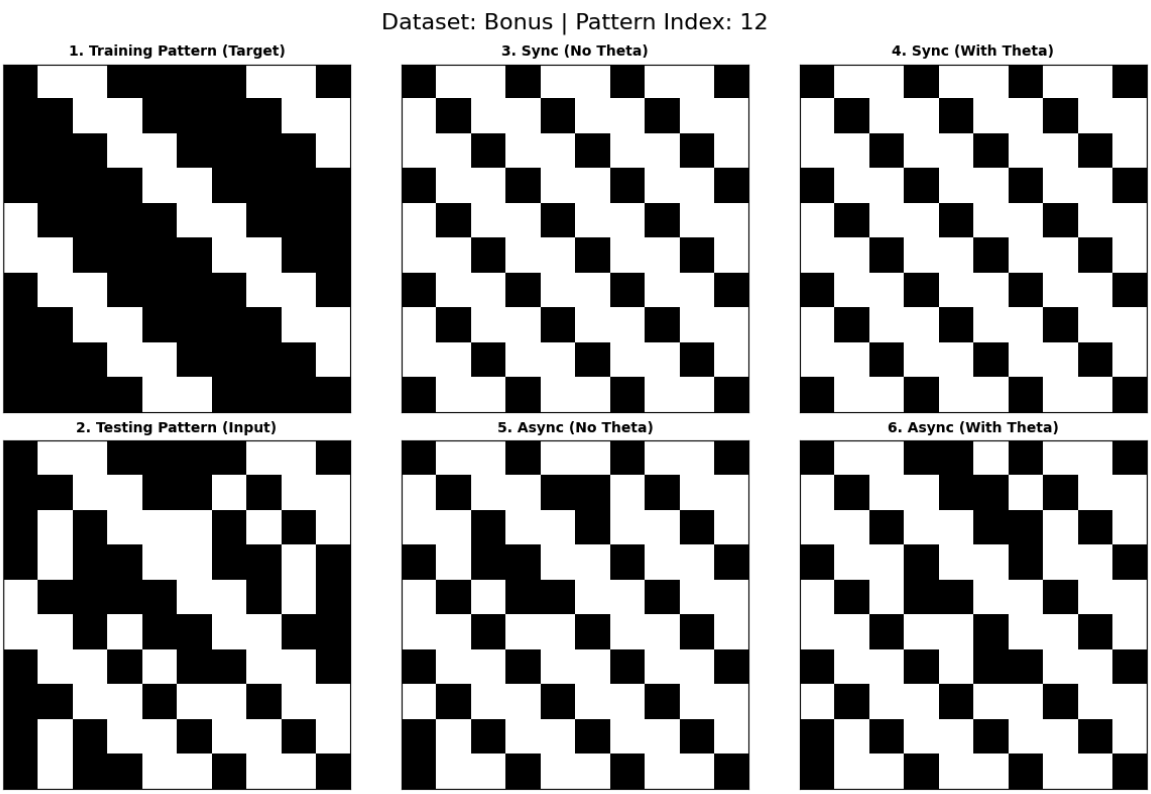
Bonus 9



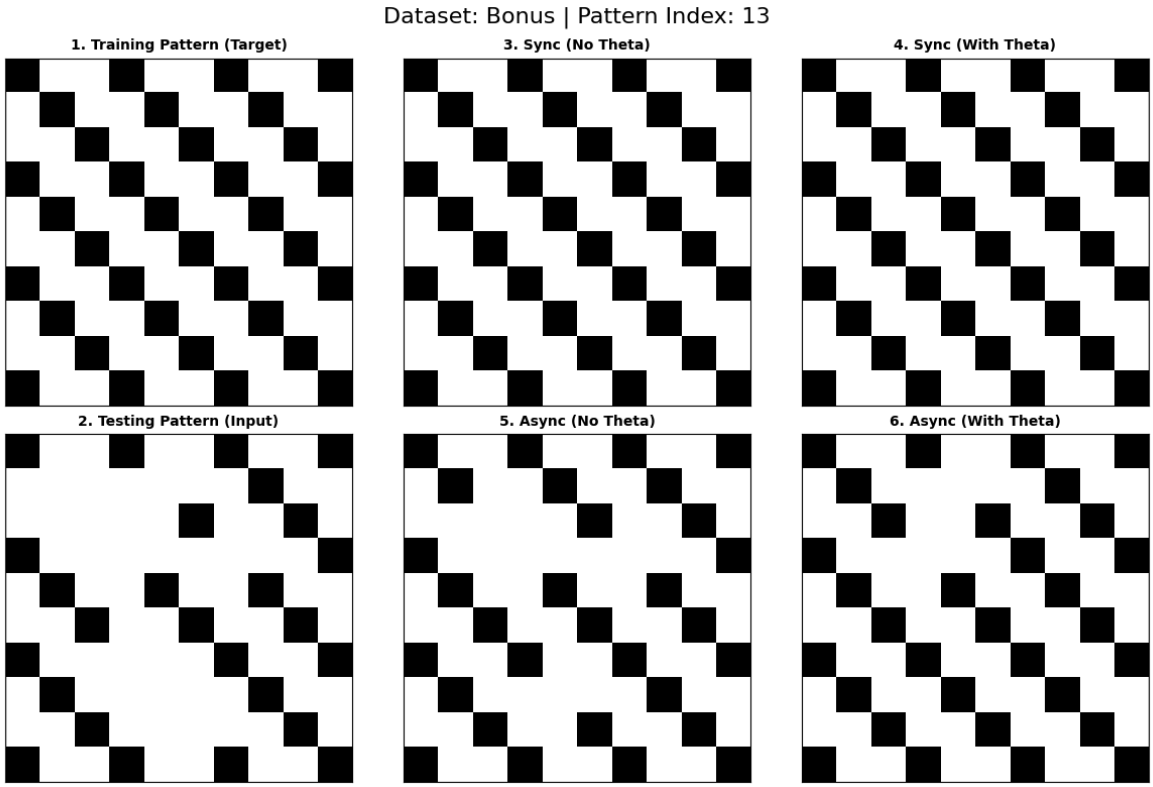
Bonus 10



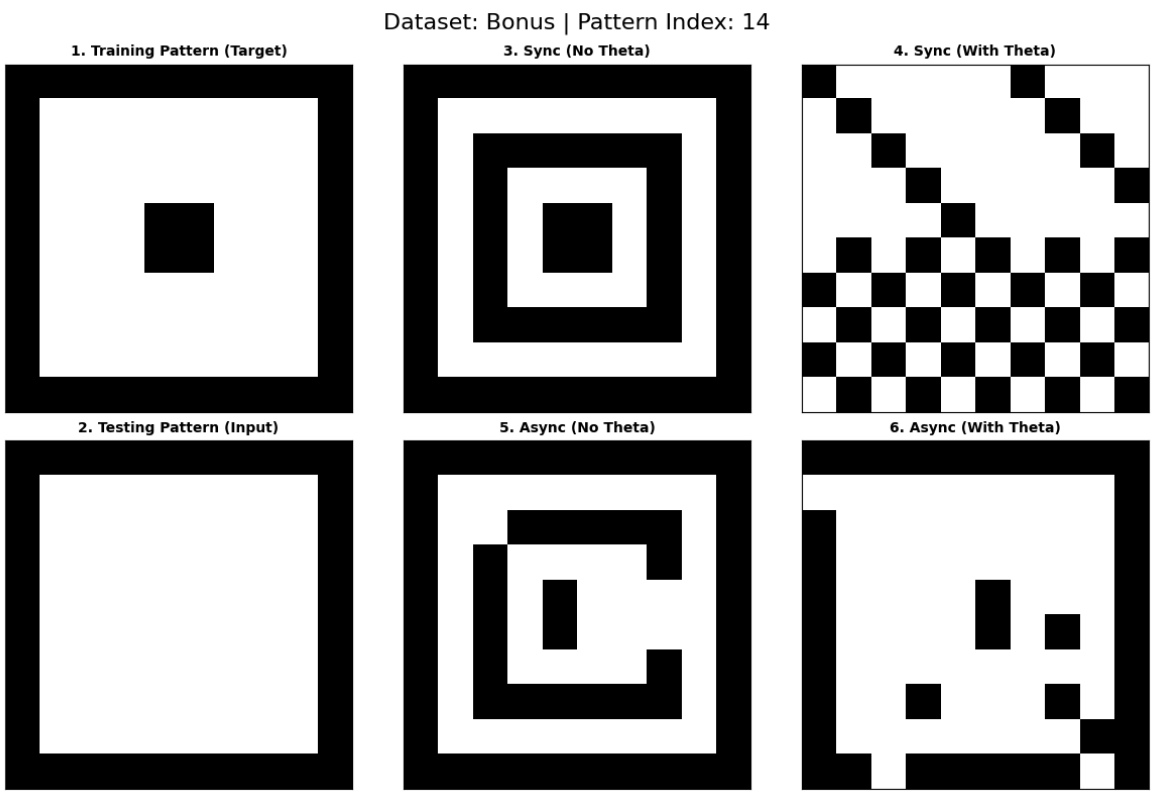
Bonus 11



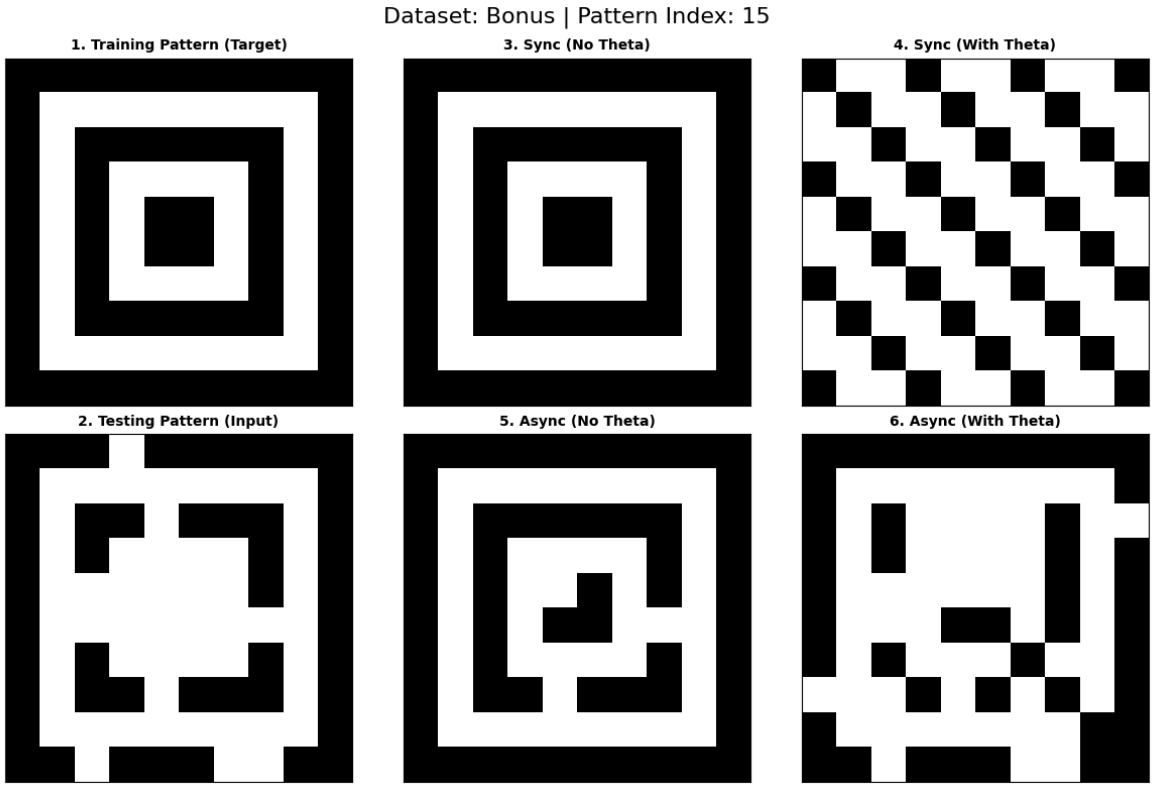
Bonus 12



Bonus 13



Bonus 14



Bonus 15

六 實驗結果分析及討論

- 同步在記憶數量少的情況下幾乎可以完美回憶, 但可能會出現 Bonus Pattern 6 這種完全回憶錯誤的情況
- 非同步在大部分情況圖形都會有缺角破損的情況，但是卻沒有明顯圖形出錯的情況，在較多需要記憶的內容看起來效果更好
- Theta=0 看起來在絕大多數情況都比較差(在 Basic 時兩者完全沒差距)，但在 Bonus 且非同步時有少數效果更好，可能是要記憶的內容